

0.88.1. Des lettres en couleur : couleurTexte — un mot en bleu

Le 0.88 avec une autre couleur et un mot entier : couleurTexte(0, 0, 31), puis BONJOUR.

1. Ce qu'il faut comprendre

C'est le 0.88, avec un mot et du bleu : couleurTexte(0, 0, 31) (seulement du bleu), puis texte(6, 8, "BONJOUR").

Toutes les lettres du mot prennent la couleur : c'est la même encre pour tout le texte.

Les fonctions de cette leçon — et le tuto de chacune

- #include <couleurTexte> → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- #include <texte> → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **337 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         30 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         96 o ← CopierTuiles - 6 lettres sur 43
"BONJOUR"      7 o ← texte() - un texte gravé
```

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
couleurFond() × 1    12 o sur place, dans main()
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <couleurTexte> // choisit la couleur des lettres
#include <texte>         // écrit un texte à l'écran
```

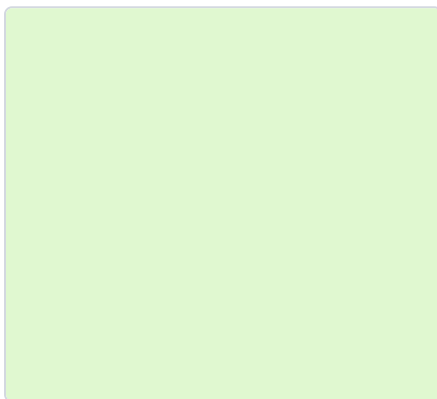
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(0, 0, 31); // rouge 0, vert 0, bleu 31 : du bleu
    texte(6, 8, "BONJOUR"); // tout le mot, en bleu

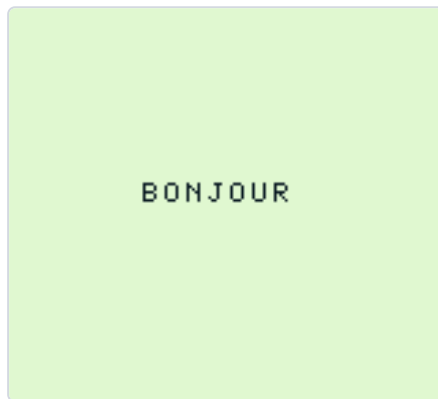
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

BONJOUR en bleu, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BONJOUR en bleu : 0, 0, 31 (0,0,31)

Ensuite : **0.88.2. Des lettres en couleur : couleurTexte** — **changer en route** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++